middleware that returns responses. Finally, you'll learn how to turn your custom middleware into a simple endpoint, using endpoint routing.

In chapter 10 you learned about strongly typed configuration using the `IOptions<T>` pattern, and in section 31.2 you'll learn how to take this further. You'll learn how to use the `OptionsBuilder<T>` type to fluently build your `IOptions<T>` object with the builder pattern. You'll also see how to use services from DI when configuring your `IOptions` objects—something that's not possible using the methods you've seen so far.

We stick with DI in section 31.3, where I'll show you how to replace the built-in DI container with a third-party alternative. The built-in container is fine for most small apps, but your `ConfigureServices` function can quickly get bloated as your app grows and you register more services. I'll show you how to integrate the third-party Lamar library into an existing app, so you can use extra features such as automatic service registration by convention.

The components and techniques shown in this chapter are more advanced than most features you've seen so far. You likely won't need them in every ASP.NET Core project, but they're good to have in your back pocket should the need arise!

31.1 Customizing your middleware pipeline

In this section you'll learn how to create custom middleware. You'll learn how to use the `Map`, `Run`, and `Use` extension methods to create simple middleware using lambda expressions. You'll then see how to create equivalent middleware components using dedicated classes. You'll also learn